Environment Setup Guide

1. Before You Start

This guide gets your machine, your Paystack test account and your repository ready for the Payment Service project. Finish every step before you start the task document. Test mode uses fake money only, and you do not need a registered business, CAC documents or a bank account.

2. Paystack Test Account

Step 1: Create an account

1. Go to **paystack.com** and click **Create free account**.
2. Enter your name, email, phone number, a business name (anything, for example "Payments Practice"), your country and a password.
3. Open the verification email from Paystack and confirm your address.

Step 2: Stay in test mode

1. Check the **Test Mode** toggle or label at the top of the dashboard. New accounts start in test mode.
2. Skip any "Activate your business" or compliance checklist. It is only for live payments; do not submit it.

Step 3: Copy your test secret key

1. Go to **Settings > API Keys & Webhooks**.
2. Copy the **Test Secret Key**. It starts with sk_test_. You will not need the public key, because customers pay on Paystack's hosted checkout page.

Keep this key on your server only. Paystack also uses it to sign webhooks, so anyone who has it can send fake "payment succeeded" events to your service. Never put it in frontend code, share it in chats or commit it. If it leaks, generate a new one from the same page.

3. Local Configuration

Step 4: Generate your service secrets

Your service needs two secrets of its own: an API key that Marketly uses to call you, and a secret for signing the callbacks you send to Marketly. Generate each with:

```
node -e "console.log(require('crypto').randomBytes(32).toString('hex'))"
```

Step 5: Create your environment file

Create .env in the root of your project:

```
PORT=4000
DATABASE_URL=postgresql://user:password@localhost:5432/payments
PAYSTACK_SECRET_KEY=sk_test_xxxxxxxx
MERCHANT_API_KEY=<generated value>
CALLBACK_URL=http://localhost:5000/callbacks
CALLBACK_SECRET=<generated value>
PAYMENT_TIMEOUT_SECONDS=1800
TEST_MODE=true
```

Commit a .env.example with the same variable names and placeholder values, so the instructor can see what your service needs. CALLBACK_URL points at whatever stub you use to receive callbacks while developing; the instructor will replace it during review.

4. Receiving Webhooks Locally

Paystack cannot send webhooks to localhost, because its servers cannot reach your laptop. A tunnel gives your machine a temporary public URL that forwards traffic to a local port.

Step 6: Start a tunnel

1. Install ngrok from **ngrok.com** and create a free account.
2. Link ngrok to your account: ngrok config add-authtoken <your-token>
3. Start your service on port 4000, then run ngrok http 4000.
4. Copy the https://...ngrok-free.app address it prints and keep the terminal open.

   On the free plan the address changes every time ngrok restarts. Repeat Step 7 whenever that happens.

Step 7: Set your test webhook URL

1. Go to **Settings > API Keys & Webhooks**.
2. In **Test Webhook URL**, enter your tunnel address followed by your route, for example https://abc123.ngrok-free.app/webhooks/paystack.
3. Leave **Test Callback URL** empty; your service sends a callback_url with each transaction.
4. Click **Save changes**.

Paystack allows one test webhook URL per account, so every test-mode event is sent to this address.

5. Verifying the Setup

Step 8: Make a test card payment

Save the following as paystack-check.mjs and run node paystack-check.mjs:

```
const res = await fetch("https://api.paystack.co/transaction/initialize", {
 method: "POST",
 headers: {
  Authorization: `Bearer ${process.env.PAYSTACK_SECRET_KEY}`,
  "Content-Type": "application/json",
 },
 body: JSON.stringify({
  email: "test@example.com",
  amount: 500000,                    // NGN 5,000 in kobo
  reference: "setup_check_" + Date.now(),  // must be unique
  callback_url: "https://example.com/done",
  channels: ["card", "bank_transfer"],
 }),
});
console.log(await res.json());
```

1. Open the authorization_url in the output.
2. Pay with card **4084 0840 8408 4081**, CVV **408**, any future expiry date.

3. Confirm that a charge.success request reached your tunnel. ngrok shows every request at http://localhost:4040, even if your route does not exist yet.

Then confirm the payment directly with Paystack. data.status should be success.

```
curl https://api.paystack.co/transaction/verify/<reference> \
 -H "Authorization: Bearer $PAYSTACK_SECRET_KEY"
```

Step 9: Make a test bank transfer

1. Run the script again and open the new checkout link.
2. Choose **Pay with Transfer**. Paystack displays a test account number and amount.
3. Open the demo bank app linked from Paystack's **Test Payments** documentation page and send the exact amount to that account.
4. Wait for charge.success. Transfers can take longer than cards; your service will have to handle that delay.

Step 10: Find your payments in the dashboard

- **Transactions:** every test payment with its amount, channel (card or transfer), status and reference. Open one to see its timeline and a **Refund** button.
- **Refunds:** the status of each refund (pending, processed or failed).
- **ngrok inspector** (http://localhost:4040): the raw webhook requests and the responses your service returned. Its **Replay** button resends a request, which is useful for testing duplicate webhooks.